

CANDY DIALOGUE ENGINE

Database Guide

1.0.0

Candy DE's database is essentially the [Candy_Database.gd](#) script. It contains dictionaries and arrays for custom data you want to make available to the engine.

To begin, open [Candy_Database.gd](#), located in the [Candy_DE/Scripts folder](#).

Actors

Let's start at the bottom of the script, with the very last dictionary: [actors](#).

```
var actors = {  
  "Alice": {  
    "Short Name": "Alice",  
    "Display Name": "Alice Turner",  
    "Speaker BBCode": ["", ""],  
    "Gender": "F",  
    "Age": 18,  
    "Disposition": "",  
    "Portrait": 0,  
    "Bubble Exempt": false,  
    "VN Join Effect": "",  
    "VN Move From Effect": "",  
    "VN Move To Effect": "",  
    "VN Leave Effect": "",  
    "LLM": 0,  
    "TTS": 0,  
    "Box Text Color": null,  
    "Box Speaker Color": null,  
    "Bubble Text Color": null,  
    "Bubble Speaker Color": null,  
    "Subtitle Text Color": null,  
    "Subtitle Speaker Color": null,  
    "Chat Text Color": null,  
    "Chat Speaker Color": null,  
  },  
}
```

The [actors](#) dictionary contains all the information Candy DE may need for actors involved in dialogues.

By default, it includes a few example entries you can remove after completing the Basic Dialogue Guide tutorial. You may want to keep **"Alice"** as an example template, but feel free to change her key name if you wish to use it for one of your own actors.

Actors are the entities who speak or act during dialogues. Typically, these are characters in your game, but they can also be objects, such as a radio or an alarm, an off-screen narrator, and so on.

Actor References

In the **actors** dictionary, top-level keys represent actor references. As a reminder, when you want to indicate who the speaker of a line is, or who the target of a command is, you enter such an actor reference.

Let's now look at the sub-keys for each actor.

Short Name

The **"Short Name"** key represents the character's name in abbreviated form. It's not used or referenced anywhere by Candy DE, but you can reference a character's **"Short Name"** key in spoken text, to display it. It can be convenient when you want to display a character's first name instead of their full name (see the **Variables** guide for more information).

You can also create additional similar keys if a character has different nicknames or diminutives that are used at different times or by different characters.

Display Name

The **"Display Name"** key represents how an actor's name is written on screen. This lets you change the name of an actor during the game without having to adapt their references everywhere in dialogues.

As an example, you would use this feature to let the player select a name of their choosing for their character, or you could display "???" for a yet-unknown character until their identity is revealed.

This is also convenient for long names: instead of writing "Dark Emperor Barkus III" every time in dialogue line data, you can just write the actor reference "Emperor" and let Candy DE display the full name on screen.

And of course, it also allows multiple characters to share the same name, while being denoted by their own individual actor references.

Gender

The **"Gender"** key is an optional feature, used for gendered spoken line variants or for gendered word substitution. See the **Variants** and **Word Substitution** guides for details, but in summary, **"Gender"** is used by Candy DE when you want to display a different text depending on the actor's gender.

Disposition

"Disposition" is a quick way to make a spoken line conditional: lines can be provided one or more dispositions and, if it matches with the disposition(s) of the speaker, the line is displayed. Otherwise, the line is skipped. See the **Dispositions** guide for details.

Note that actors can be assigned multiple dispositions. In such cases, dispositions are written in a single string and must be separated by a comma (,). To avoid parsing errors, disposition names should never include special characters except underscores (i.e. letters, numbers and '_' only).

Portrait

The **"Portrait"** key indicates whether a portrait should be displayed for the actor, overriding any other settings.

A value of 0 will disable the override, -1 will prevent portraits from displaying, and 1 will always display portraits (provided the selected dialogue mode supports portraits, such as *Box*).

This can be used to disable portraits for off-screen narrators or for actors that are just objects that display sound effects in text form.

Bubble Exempt

The **"Bubble Exempt"** key is used in "Bubbles" dialogue mode (comic book-style speech bubbles above characters) when you don't want an actor to use speech bubbles. A value of 'true' will use an alternate display method, and 'false' will disable this override.

This isn't required for off-screen characters, as "Bubbles" mode automatically falls back to alternate display methods automatically. Instead, **"Bubble Exempt"** is used when an actor is in fact present on screen, but speech bubbles are not desired (e.g. objects that express sound in text form).

See the [Speech Bubbles](#) guide for more information.

VN Effect

The various **"VN Effect"** keys allow specifying a default animation for an actor, when the actor is assign to a Visual Novel bust. See the [Visual Novels](#) guide for details.

LLM

The **"LLM"** key determines whether the actor's spoken text should be generated by an LLM (AI), overriding default settings elsewhere.

A value of 0 disables the override, -1 means the actor's speech will never be generated with an LLM, and 1 means the actor's speech will always be generated by an LLM.

Of course, LLM generation requires that you let your game connect to a text generation API. See the [LLM](#) guide for details.

Text Color

The various **"Text Color"** keys override the color of the actor's spoken text. See the [Text Color](#) guide for details.

Speaker Color

The various **"Speaker Color"** keys override the color of the actor's name when speaking. See the [Text Color](#) guide for details.

Player References

Sometimes, Candy DE needs to know whether an actor represents the player, or one of multiple player characters.

This is used, for example, when portrait display settings differ between NPCs and player characters, or for locating player nodes in the game world when their paths differ from NPC nodes.

Simply add any such actor references from the **actors** dictionary to the **player_refs** array, either directly in code or dynamically at runtime:

```
var player_refs = ["Player", "player"]
```

Actor Roles

Roles are used when you don't know in advance which actor will speak a line or should be the target of a command. This is the case, for example, for player followers.

Roles are simple to use: add them to the **roles** dictionary, and assign actor references to them at runtime, based on game events.

```
var roles = {  
  "°enemy_1": "",  
  "°companion_1": "Tarana",  
}
```

Note that role names must be prefixed with the **role_symbol** (default "°") so Candy Dialogue Engine can recognize them as such.

Flags

The **flags** dictionary can be used to store progression flags for your game and for dialogues. The benefit of using this dictionary over one of your own is that the \$Flag command directly targets it, letting you just type a flag name instead of using the full variable reference syntax.

```
var flags = {  
  "Test": 0,  
  "flag": 0,  
  "quest_started": false,  
}
```

If you prefer to use a different dictionary, you can make the \$Flag command work with it by pointing the flags dictionary to it, like this:

```
var flags = your_script.your_flag_dictionary
```

Dialogue Variables

The **variables** dictionary is used for quickly referencing variables you often use in dialogues. This isn't required, just quicker than typing full variable references in dialogue lines or spoken text.

```
var variables = {  
  "variable": "value",  
  "player": "Revan",  
  "test": null  
}
```

Lexicon

The **lexicon** (sometimes called a glossary in other dialogue systems) is a dictionary of keywords that can be automatically given a special format in spoken text, can display a tooltip when hovered with the mouse, and can be made clickable with custom effects.

```
var lexicon = {  
  "*": {  
    "Alice": {  
      "Display": "",  
      "BBCode": ["[b][color=blue]", "[/color][b]"],  
      "Tooltip": "Alice is a student.",  
      "Clickable": false,  
      "Click_Data": [],  
    },  
  },  
}
```

Each keyword can have its own display and tooltip text, BBCode formatting and click behavior.

See the [Lexicon](#) guide for more information.

Substitution Table

The **substitution_table** dictionary is used by Candy DE's word substitution feature.

```
var substitution_table := {  
  "Default*": {  
    "he, she": {  
      "auto",  
      {"Gender": {  
        "M": "he",  
        "F": "she",  
      }},  
    },  
  },  
}
```

The dictionary contains the various substitution rules you want to create, sorted by variant. For details, see the [Word Substitution](#) guide.

Creator Data

At the top of the script, you'll notice three variables:

```
var creator_disposition_list = []  
  
var creator_actor_list = []  
  
var creator_role_list = []
```

These variables are used to provide reference data to the companion Candy Dialogue Creator.

By default, Candy DC automatically reads from your project's **actors** and **roles** dictionaries. It then generates drop-down lists of actors and roles to make data entry faster when writing lines. However, depending on your project, these dictionaries can contain a large number of entries, which can make Candy DC's lists cumbersome.

To keep things manageable, you can instead point Candy DC to **creator_actor_list** and **creator_role_list**. You can then fill these arrays manually with only the actors and roles you reference most frequently, such as your main characters or recurring roles. You'll still be able to use other actors and roles in Candy DC: you'll just need to type them manually.

In addition, Candy Dialogue Engine doesn't have a dedicated dictionary for dispositions, so Candy DC is pre-configured to read directly from **creator_disposition_list**. If your game uses dispositions, you should manually populate that list with all dispositions you want available in Candy DC's drop-down disposition list to avoid typing dispositions manually.